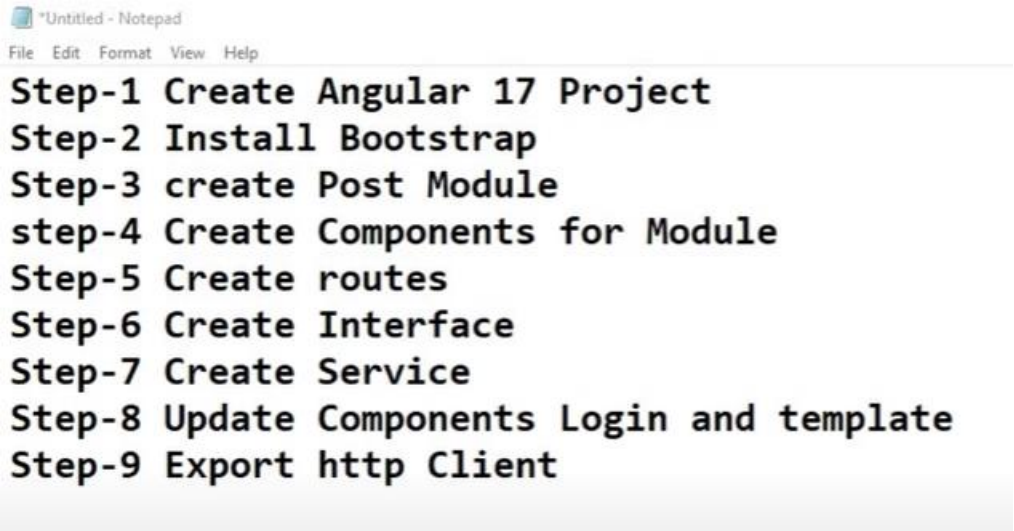
Sahosoft Solutions https://www.youtube.com/watch?v=3syu_UfCrWM&list=PL_qizAfcpJ-OzjbgGv9xobErKY7v9o0qi&index=1&pp=iAQB

npm install -g @angular/cli

ng new projectName

ng g c componentName



Error: Could not find the '@angular-devkit/build-angular:dev-server' builder's node package.

npm install --save-dev @angular-devkit/build-angular

topic: preserveWhitespaces
topic: viewprovider
topic : Encapsulation (for css style)
topic : Parent to child Communication With metadata
topic: child to parent Communication With metadata
topic:Directives (component, styructural,attribute)
topic: @If, elseif and else in Angular 17 | Structural Directives
topic: @switch and @case in Angular 17 | Structural Directives
topic: @for, track, $even, $odd, $first, $last, $count
topic: Attribute Directives
TOPIC: ngStyle Directives in Angular 17

TOPIC: ngClass Directives in Angular 17
TOPIC: Data Binding in Angular 17
TOPIC:interpolation Binding in Angular 17
Topic: Singleton Service in Angular 17
Topic : tree shaking in Service in Angular 17
Topic : how router works in Angular 17
Topic: **route configuration in Angular 17**
Topic: wildcard route in Angular 17
Topic : how to set base URL in Angular 17
Topic: Auxiliary Routes in Angular 17 | Named and Multiple Router-Outlets
Topic: Routes Guards in Angular 17 | Route Guards (Need to login first when accessing any route)
Topic: Auth Guard using Login page in Angular 17 | canActivate Route Guard

<u>Generate Guard:</u>

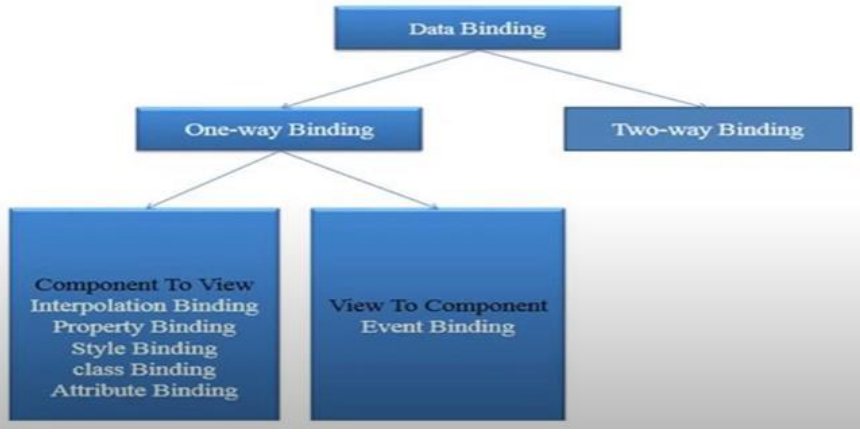
ng g g guardName

```
<div style="text-align: center;">
<h1>Welcome to Sahosoft Technologies..</h1>


<!-- {{employees}} -->
@for(emp of employees;track emp.id){
    <ul>
        <li>{{emp.id}}</li>
        <li>{{emp.name}}</li>
    </ul>
}


</div>
```

```
app > app.component.html > div > table > tr > td
    <table border="1">
        <tr>
            <th>EMP ID</th>
            <th>EMP NAME</th>
        </tr>
        @for(emp of employees;track emp.id){
            <tr>
                <td>{{emp.id}}</td>
                <td>{{emp.name}}</td>
            </tr>
        }@empty{
            <tr>
                <td colspan="2">
                    No Data Found
                </td>
            </tr>
        }

    </table>
```

```
BLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                node +
tial Chunk Files | Names | Raw Size
```

Data Binding



Route Guards

There are 5 types of Route Guards we use in Angular.

CanActivate
This helps to decide whether the route can be activated or not.1
CanActivateChild
This helps to decide whether child routes can be activated or not.
CanDeactivate
This helps to decide whether the route can be deactivated or not. Generally, this is used to warn a user if they are navigating from the component.
Resolve
This performs route data retrieval before any activation of the route.
CanLoad
Checks to see if the user can navigate to the module which is lazily loaded.

Route Guards

In Short:

- ✓ CanActivate guard (e.g. it checks route access).
- ✓ CanActivateChild guard (checks child route access).
- ✓ CanDeactivate guard (prompt for unsaved changes).
- ✓ Resolve guard (pre-fetching route data).
- ✓ CanLoad guard (check before loading feature module).

Interview Questions

Promises and Observables are both used in Angular to handle asynchronous operations, but they have key differences:

Feature	Promise	Observable
Purpose	Handles a single asynchronous event that will resolve or reject once.	Handles a stream of asynchronous events over time, which can emit multiple values, errors, or completion signals.
Execution	Executes immediately upon creation.	Executes only when subscribed to.
Cancellation	Cannot be cancelled once started.	Can be cancelled by unsubscribing.
Error Handling	Uses `.catch()` to handle errors.	Uses the `error` callback in the `subscribe` method.
Use Cases	Suitable for one-time asynchronous operations like HTTP requests where you expect a single response.	Ideal for handling continuous data streams, event handling, and scenarios requiring multiple values over time, such as user input or real-time updates.
Operators	Limited built-in operators.	Offers a wide range of operators for transforming, filtering, and manipulating data streams.
Library	Built-in to JavaScript.	Requires RxJS library.

Promises are simpler and suitable for basic asynchronous tasks, while

Observables provide more flexibility and power for complex scenarios involving multiple data emissions and the need for cancellation and data manipulation.